

XSS 漏洞报告 (Lab4: 将 XSS 映射到 HTML 上下文中, 大部分标签和属性被屏蔽)

1. 漏洞名称

跨站脚本攻击 (XSS) — WAF 绕过与自动化触发

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站搜索功能存在反射型 XSS 漏洞, 但部署了 WAF (Web 应用防火墙) 来拦截常见的 XSS 标签和事件。攻击者通过 Burp Suite 的 Intruder 模块, 结合 XSS 速查表, 以“暴力枚举”的方式进行黑盒测试, 成功绕过了 WAF 的过滤规则。最终使用未被过滤的 body 标签和 onresize 事件, 配合 iframe 负载攻击构造出自触发 Payload, 无需用户交互即可执行恶意代码。

4. 漏洞位置

请求方式: GET /search

漏洞参数: search

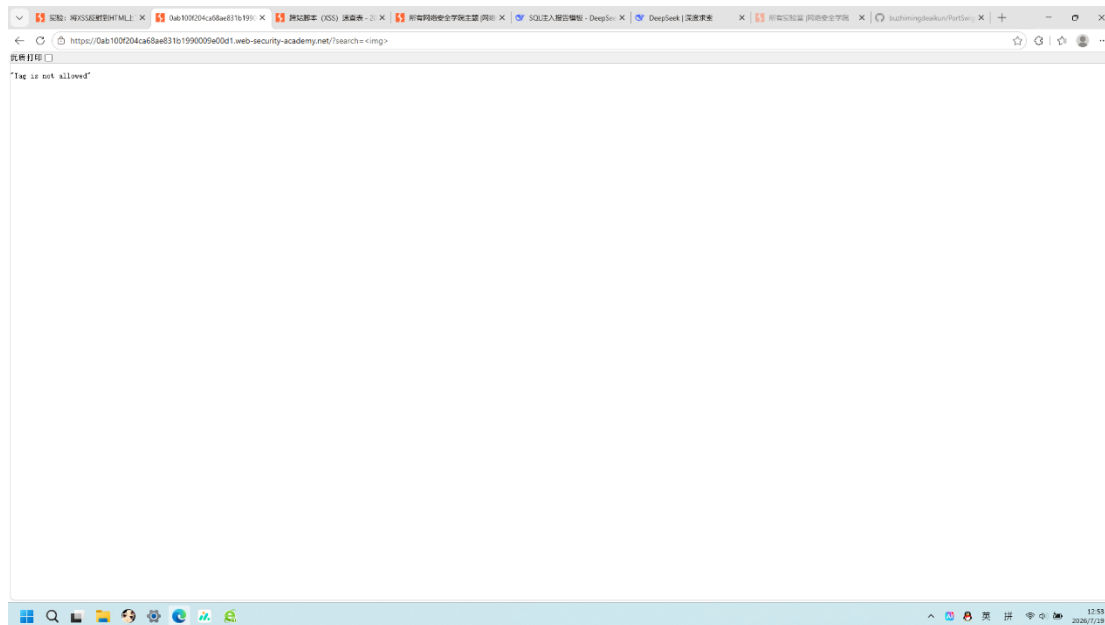
5. 漏洞复现过程

第一阶段: 黑盒测试与 WAF 规则探测

第一步: 拦截搜索请求, 准备注入

打开 Burp 浏览器, 使用实验室搜索功能, 拦截生成的 GET 请求, 并发送至 Burp Intruder。

图 1: Burp Proxy 拦截搜索请求

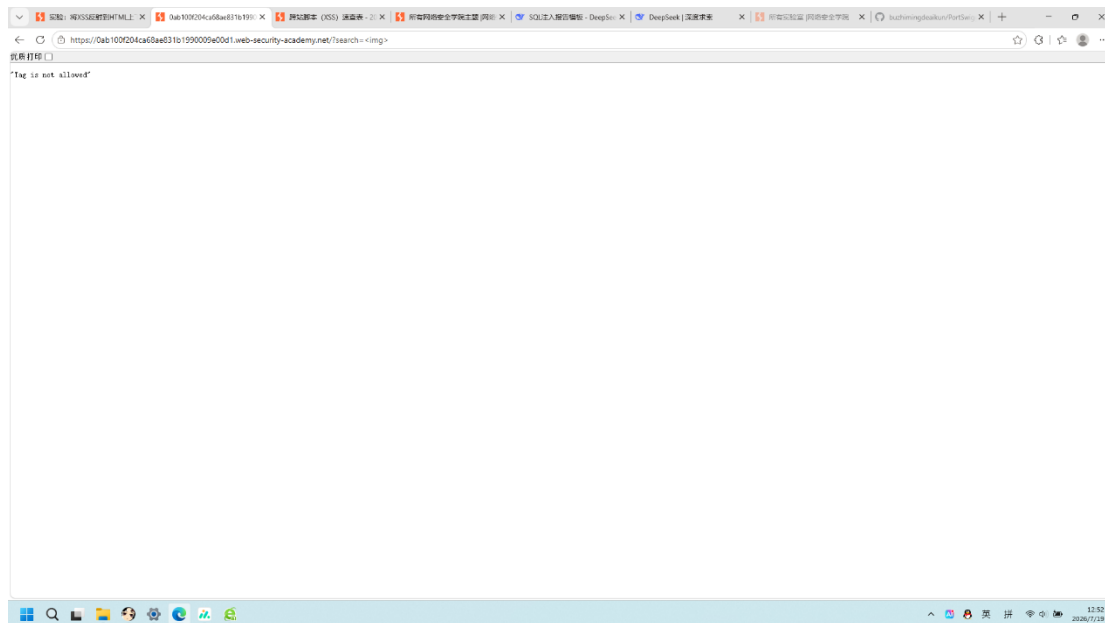


说明：找到搜索功能的请求包，准备进行 Payload 测试。

第二步：访问 XSS 速查表，准备攻击向量

为了测试各类标签，访问 PortSwigger 官方 XSS 速查表获取标准攻击向量库。

图 2：访问 PortSwigger XSS 速查表

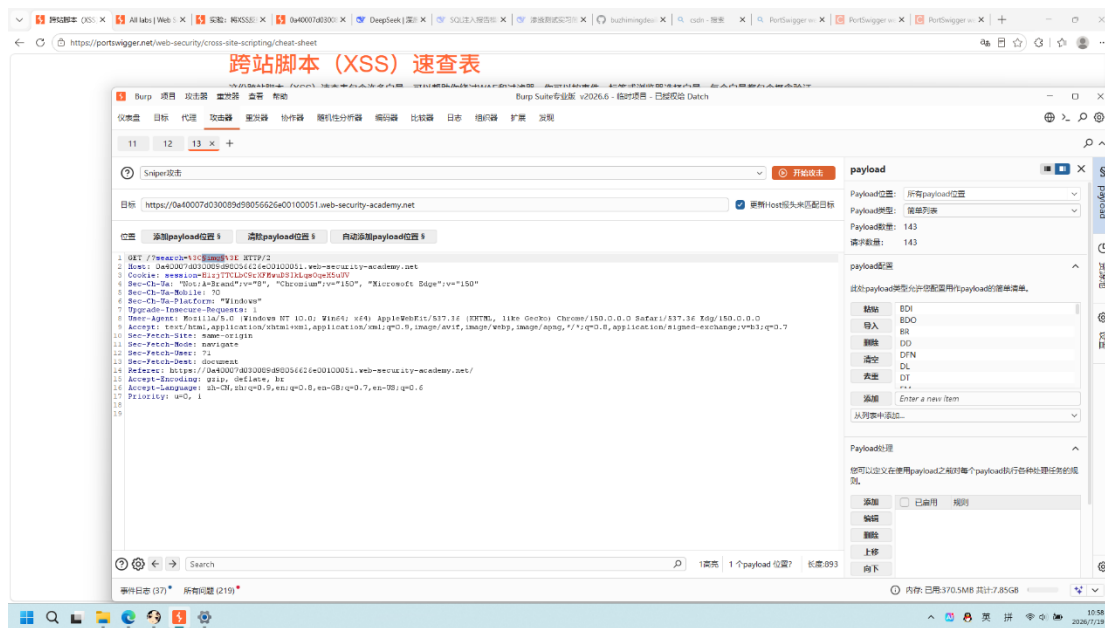


说明：速查表涵盖了所有可能绕过 WAF 的 HTML 标签和事件属性。

第三步：Burp Intruder 配置“标签”爆破

将搜索词值替换为 <>，在中间添加 Payload 位置。从速查表复制所有 HTML 标签列表，粘贴到 Intruder 的 Payload 配置中。

图 3: Burp Intruder 配置标签列表爆破



说明: 准备测试哪些 HTML 标签没有被 WAF 拦截。

图 4: 标签爆破结果, 发现 body 标签未被拦截

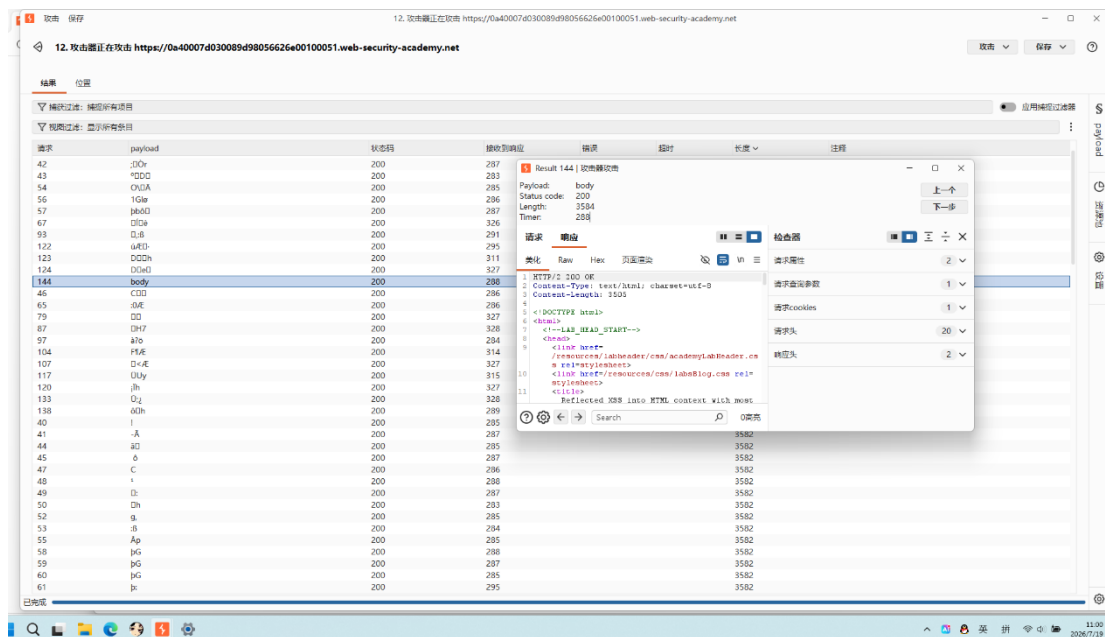


说明: 测试发现 WAF 拦截了大部分标签, 但 body 标签成功通过, 说明 body 标签可用。

第四步：手工测试验证标签状态

直接在浏览器搜索框输入 `<body>`，验证其可被正常解析。

图 5: 搜索框手工验证 body 标签



说明：手工确认 WAF 对 body 标签放行。

第五步：手工测试常规属性拦截

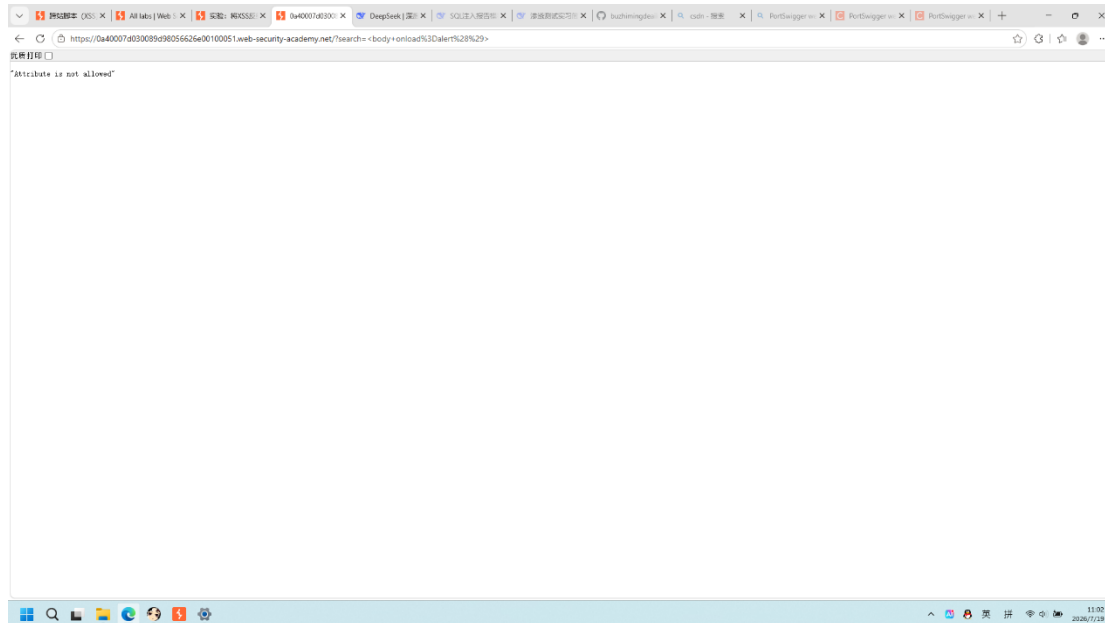
在搜索框中输入带有常见事件处理器的标签，验证 WAF 的拦截机制。

图 6：常规 Payload 被 WAF 拦截 (Tag is not allowed)



说明：WAF 会直接拦截非白名单内的标签。

图 7：onload 属性被拦截 (Attribute is not allowed)



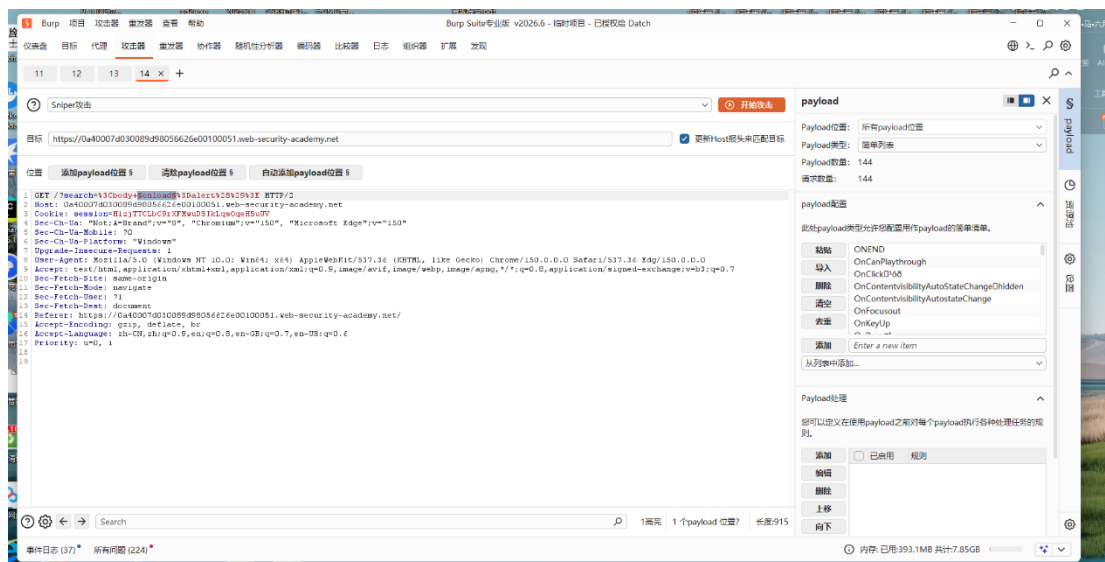
说明：WAF 会禁止使用 onload 等高度敏感的事件处理器。

第二阶段：事件处理器探测与绕过

第六步：Burp Intruder 配置“事件”爆破

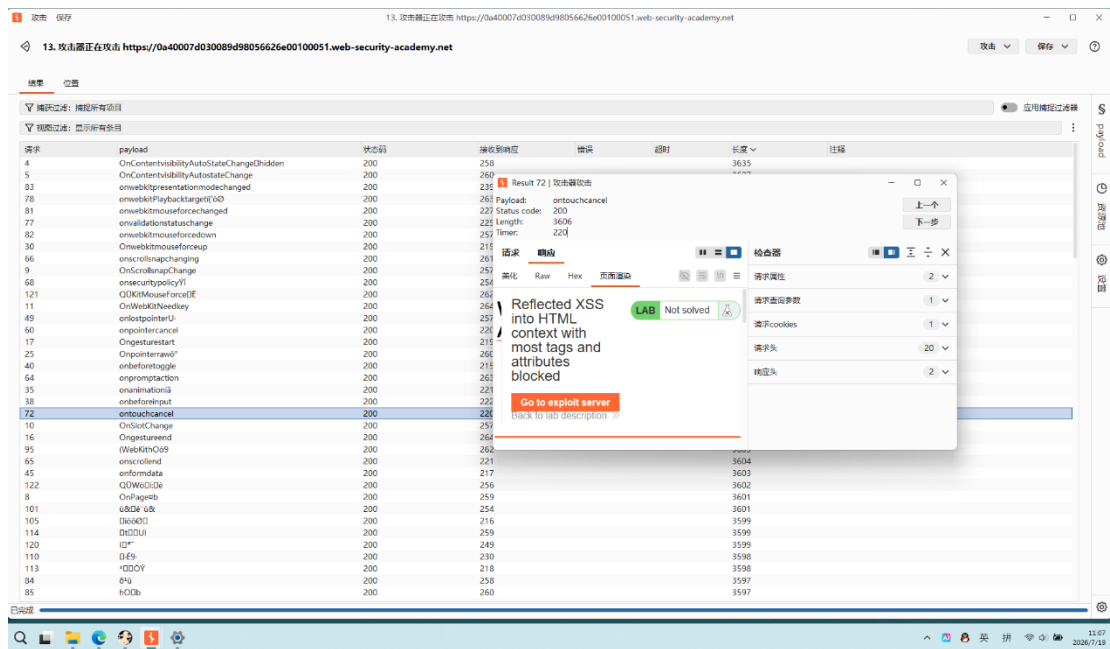
确定 body 标签可用后，将搜索词修改为 <body=1> 并添加 Payload 位置。从 XSS 速查表中复制所有事件属性列表，替换 Intruder 的 Payload 列表，进行第二轮爆破。

图 8：Burp Intruder 配置事件属性爆破



说明：准备测试在 body 标签上，哪些 JS 事件属性没有被 WAF 拦截。

图 9：事件爆破结果，发现 onresize 未被拦截



说明：成功找出 onresize 事件可以顺利通过 WAF 检测。

第三阶段：手动验证与组合 Payload

第七步：测试组合 Payload

在搜索框中输入探测出的有效组合 `<body onresize=print()>` 进行手动测试。

图 10：搜索框手动测试有效 Payload



说明：该组合可以绕过 WAF 并返回 200，但因为 onresize 需要浏览器窗口大小发生变化才能触发，直接访问不会弹窗。

图 11：打印预览确认 Payload 生效



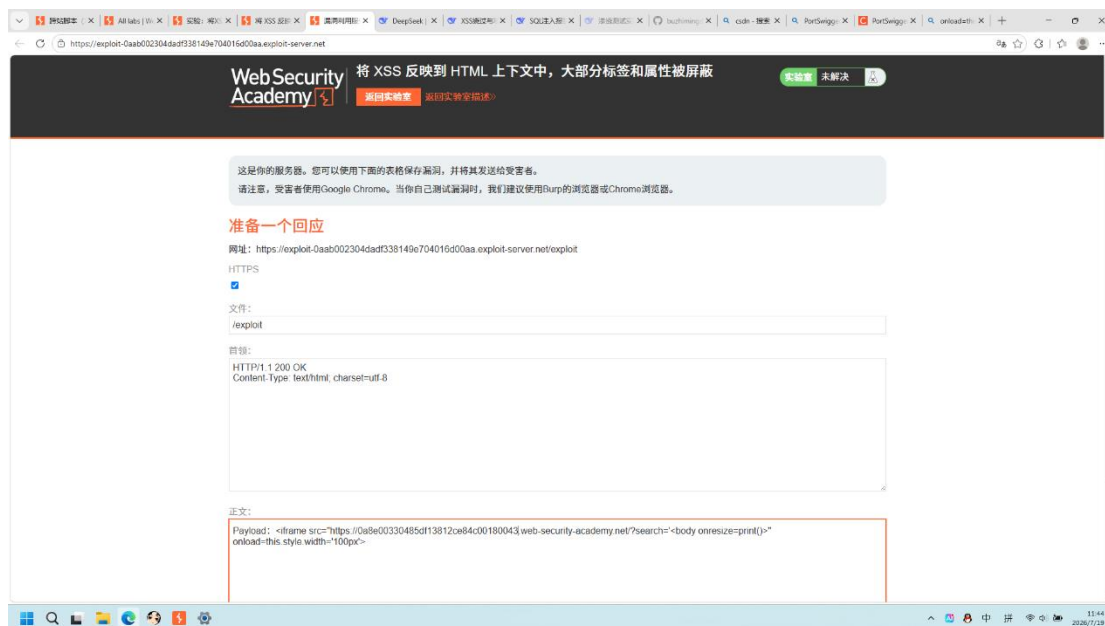
说明：通过浏览器的打印预览功能，可以确认 Payload 确实被页面 HTML 代码正确渲染。

第四阶段：自动化触发（利用 iFrame 负载攻击）

第八步：构造无交互触发机制

为了解决“无需用户交互”的问题，需要构造一个包含 iframe 的 Web 页面。利用 iframe 的 onload 事件在加载瞬间改变宽度，从而触发内部 body 的 onresize 事件。这里使用了另一个测试过的有效 Payload ontouchcancel 或其替代方案。

图 12：在 Exploit Server 中编写自动触发脚本



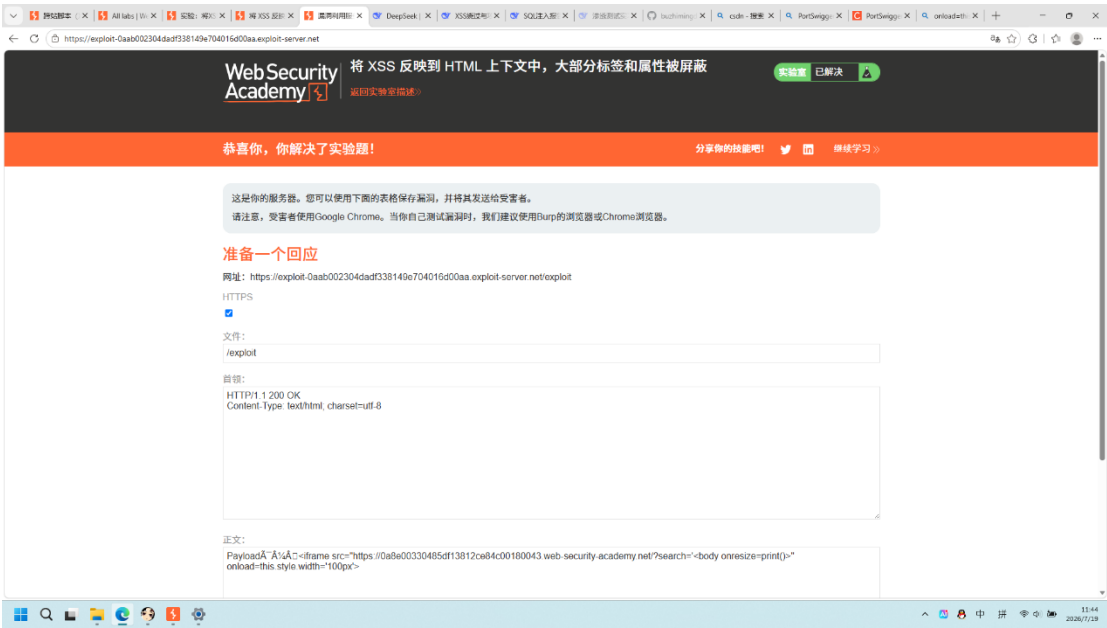
说明：当受害者访问该页面时，iframe 载入的一瞬间 width 变为 100px，随之触发子

页面 body 的尺寸变化，导致 onresize 事件被触发。

第九步：存储与交付攻击

点击 Exploit Server 页面的“Store”按钮保存恶意页面，并点击“Deliver exploit to victim”发送给受害者。

图 13：实验最终解决



说明：成功模拟受害者浏览过程，自动化触发 XSS，实验大功告成！

6. 漏洞原理

这是一个复杂的 WAF 规则探测与组合绕过 案例：

1. **WAF 失效**：WAF 使用黑名单机制拦截常见 XSS 标签和属性，但未能覆盖 body 标签与 onresize 事件的组合。
2. **事件触发技巧**：onresize 需要用户手动调整窗口大小才会触发。实验破解的关键在于“iFrame 负载攻击”：通过外层 iframe 标签在 onload 瞬间修改自身尺寸，强行触发内部 body 的尺寸变化，从而自动执行 onresize=print()。
3. **漏洞本质**：这属于反射型 XSS 的利用，但其执行流程被巧妙地转化为了“一次点击、触发所有”的定向攻击。

7. 漏洞影响

- **防守规避**：攻击者可通过 Fuzzing（模糊测试）找出 WAF 配置的漏洞，绕过企业的防护体系。
- **无交互攻击**：利用 iframe + onresize 的攻击向量，用户甚至不需要点击任何地方，只要访问恶意网页就会中招，极大提高了攻击成功率。

8. 修复建议

- ① **使用白名单机制（推荐）**：WAF 不应仅依赖黑名单，应配置严格的 HTML 标签和属性白名单，只允许业务必须的标签存在。
- ② **全面的 HTML 编码**：后端在输出搜索词到页面时，必须对用户输入做严格的 HTML 实体编码（将 <, > 转义为 <, >）。如果进行了正确的输出编码，无论 WAF 是否绕过，XSS 都将无效。
- ③ **事件处理防护**：尽量避免在内联标签中使用 on* 事件处理器。

9. 工具

- **Burp Suite**（Intruder 模块进行黑盒探测与爆破）
- **XSS Cheat Sheet**（PortSwigger 官方 XSS 攻击向量速查表）
- **PortSwigger Web Security Academy**（Exploit Server 漏洞利用服务器）